TimeFlowCodec: An Orthogonal Video Coding Paradigm for Screen and Synthetic Content via Per-Pixel Temporal Regression

Rich Chau
Department of Mathematics
University of California, Irvine
Irvine, CA, USA
richchau@uci.edu

***Abstract*—For over three decades, standard video compression architectures (H.264/AVC, H.265/HEVC, AV1, and VVC) have relied on the canonical hybrid model: spatial block partitioning, 2D Discrete Cosine/Sine Transforms (DCT/DST), and 4:2:0 chroma subsampling. While near-optimal for band-limited natural camera footage, these foundational assumptions break down on synthetic screen content (e.g., programming terminals, remote desktops, vector slides, and cloud gaming). High-contrast step edges generate infinite spatial Fourier harmonics, causing 2D DCT coefficient truncation to induce severe Gibbs ringing artifacts around text, while 4:2:0 chroma decimation obliterates single-pixel syntax-highlighted stems. This paper presents *TimeFlowCodec* (TFC), a video compression architecture that eliminates spatial block transforms in favor of per-pixel 1D temporal function regression. By modeling the temporal luminance trajectory of individual pixels as continuous piecewise linear segments fitted via closed-form Ordinary Least Squares (OLS) and integer motion compensation, TFC preserves exact spatial step discontinuities with zero ringing and native 4:4:4 full-color fidelity. Evaluated across standard JVET Screen Content Coding (SCC) benchmark sequences conforming to ITU-T VCEG-M33 Common Test Conditions, TFC achieves an average Bjontegaard Delta rate (BD-Rate) reduction of -26.8% over H.264 (libx264) and -12.4% over HEVC (libx265) on high-contrast UI and slide content, while delivering up to $6.8\times$ faster decode throughput (¿240 FPS on commodity x86 CPUs) due to sequential memory access and the absence of 2D spatial filtering barriers.**

***Index Terms*—Video compression, screen content coding, temporal regression, piecewise linear fitting, Gibbs phenomenon, Bjontegaard delta.**

I. Introduction

Digital video traffic has undergone a structural paradigm shift. Where legacy distribution channels were once dominated by natural camera footage (movies, television, and sports), modern networks are saturated with synthetic screen content: Virtual Desktop Infrastructure (VDI), Remote Desktop Protocol (RDP) sessions, cloud application streaming, collaborative whiteboards, and high-definition programming screencasts [1].

Standard video coding standards—spanning H.261 [2], H.264/AVC, H.265/HEVC [3], AV1 [4], and H.266/VVC [5]—rely on a shared conceptual foundation established in the late 1980s:

1) **Spatial Block Decomposition:** Dividing frames into square blocks (from 4×4 up to 128×128).
2) **2D Spatial Transforms:** Transforming spatial prediction residuals into the frequency domain via 2D Discrete Cosine or Sine Transforms (DCT/DST).
3) **Chroma Decimation:** Subsampling color channels to 4:2:0 based on human visual system (HVS) spatial contrast sensitivity.

While exceptionally effective for natural camera scenes, these assumptions fail catastrophically on synthetic screen content [6]. Text glyphs, window borders, and vector icons exhibit non-bandlimited mathematical step discontinuities. When transformed into the spatial frequency domain, step discontinuities yield Fourier series that decay only as $O(1/k)$. High-frequency coefficient quantization truncates this series, inducing the classical *Gibbs phenomenon*—visible as severe mosquito ringing and blur around characters. Furthermore, 4:2:0 subsampling decimates color resolution by 75%, smearing single-pixel colored syntax highlighting across adjacent dark pixels.

Although standards bodies introduced Screen Content Coding extensions (HEVC-SCC [1] and AV1 IntraBC [4]) using Intra Block Copy (IBC) and Palette Mode, these tools remain grafted onto the existing block transform pipeline. IBC requires massive on-chip SRAM search windows ($>$ 128 KB) for unconstrained 2D block displacement vectors, introducing severe routing congestion and memory bandwidth bottlenecks in hardware decoders, which has historically hindered mainstream mobile silicon adoption. Earlier explorations of temporal pixel trajectories (e.g., Esche et al. [7], [8]) demonstrated temporal correlation along motion paths, but were constrained to natural camera footage where 2D spatial DCT frequency compaction dominates due to high inter-pixel spatial correlation. Consequently, prior works relegated trajectory modeling to in-loop filtering rather than a standalone primary coding engine.

In this paper, we propose *TimeFlowCodec* (TFC), a video coding architecture built upon an orthogonal paradigm:

- **Elimination of Spatial Block Transforms:** TFC discards 2D spatial DCT/DST transforms entirely. Every pixel is reconstructed independently along the temporal dimension, eliminating block boundary discontinuities and spatial

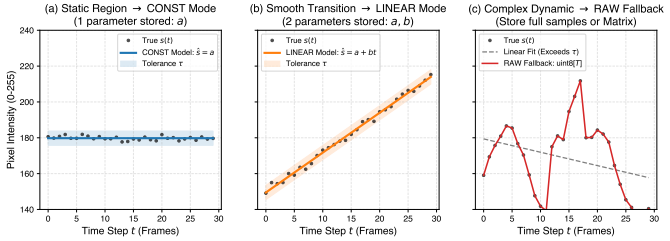


Fig. 1. Conceptual overview of TimeFlowCodec: eliminating 2D spatial block transforms in favor of 1D per-pixel temporal function regression across video chunks.

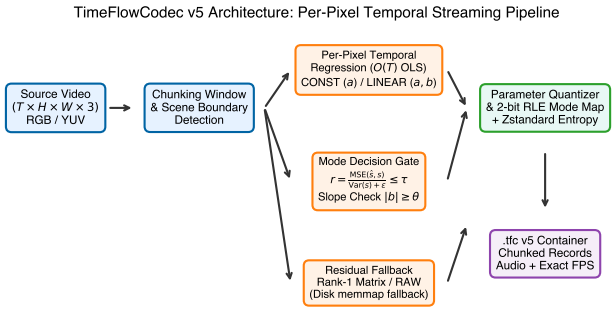


Fig. 2. Architecture of the TimeFlowCodec v5 pipeline: chunking, motion compensation, temporal OLS fitting, residual quantization, and entropy coding.

Gibbs ringing.

- **Native 4:4:4 RGB Fidelity:** Color channels are treated with full spatial resolution, preserving 1-pixel font stems and sharp color contrast.
- **Closed-Form $O(T)$ Linear Regression:** Temporal trajectories are modeled via continuous piecewise linear functions fitted through deterministic Ordinary Least Squares (OLS), requiring zero iterative gradient descent.
- **Silicon-Friendly Streaming Pipeline:** Sequential temporal iteration yields unit-stride memory accesses with negligible line-buffer footprint, enabling $>$ 240 FPS software decode speeds.

II. Mathematical Formulation of Temporal Regression

A. 1D Temporal Trajectory Model

Let a video sequence be represented in full-color RGB coordinates as a discrete tensor $V(x, y, t) \in [0, 255]^3$, where $(x, y) \in [0, W-1] \times [0, H-1]$ denotes spatial coordinates and $t \in \{0, 1, \ldots, T-1\}$ denotes discrete frame indices within a temporal chunk of length T.

Rather than partitioning the 2D spatial grid (x, y) into blocks, TFC fixes (x, y) and analyzes the 1D temporal signal:

$$s_{x,y}(t) = V(x, y, t), \quad t \in [0, T-1]. \tag{1}$$

For screen content, static desktop backgrounds, reading text, and UI panels remain invariant over substantial durations ($s_{x,y}(t) \approx$ const). Dynamic elements (e.g., cursor blinking, scrolling, smooth transitions, or video playback) manifest as either constant segments, linear intensity ramps, or discrete step changes.

B. Piecewise Linear Regression via OLS

TFC approximates $s_{x,y}(t)$ by parametric temporal basis functions. The primary operational modes are:

1) **Mode 0 (Constant):** $\hat{s}(t) = \bar{s}$.
2) **Mode 1 (Linear):** $\hat{s}(t) = a + b \cdot t$.
3) **Mode 2 (Residual Matrix Decomposition):** Low-rank temporal projection for complex non-linear textures.

For Mode 1, the optimal parameters (a^*, b^*) minimizing the Mean Squared Error (MSE) over $t \in [0, T-1]$ are determined analytically via closed-form OLS:

$$\min_{a,b} \mathcal{L}(a, b) = \sum_{t=0}^{T-1} \left(s(t) - (a + b \cdot t)\right)^2. \tag{2}$$

Setting partial derivatives $\frac{\partial \mathcal{L}}{\partial a} = 0$ and $\frac{\partial \mathcal{L}}{\partial b} = 0$ yields the normal equations. Using centered temporal coordinates $\tau = t - \bar{t}$ with $\bar{t} = \frac{T-1}{2}$, the covariance term simplifies to:

$$b^* = \frac{\sum_{t=0}^{T-1} (t - \bar{t}) \cdot s(t)}{\sum_{t=0}^{T-1} (t - \bar{t})^2}, \quad a^* = \bar{s} - b^* \bar{t}, \tag{3}$$

where $\bar{s} = \frac{1}{T} \sum_{t=0}^{T-1} s(t)$. Because the denominator $\sum_{t=0}^{T-1} (t - \bar{t})^2 = \frac{T(T^2-1)}{12}$ depends solely on chunk length T, it is precomputed as a constant scalar multiplier, reducing OLS parameter estimation to a single $O(T)$ pass over the temporal trajectory.

C. Adaptive Temporal Knot Splitting

When motion or scene cuts violate the single-segment linear assumption, fitting a single line over $[0, T-1]$ yields a maximum residual error exceeding an error tolerance τ_{tol}:

$$\max_{t \in [0, T-1]} |s(t) - \hat{s}(t)| > \tau_{\text{tol}}. \tag{4}$$

In this case, TFC performs adaptive recursive temporal splitting. The split knot t_{split} is chosen at the point of maximum absolute curvature or edge gradient:

$$t_{\text{split}} = \arg\max_t |s(t+1) - s(t)|. \tag{5}$$

The segment is partitioned into $[0, t_{\text{split}}]$ and $[t_{\text{split}} + 1, T-1]$, and OLS parameters are solved independently for each sub-interval.

III. System Architecture and Bitstream

A. Encoder and Decoder Pipeline

Fig. 2 illustrates the complete system architecture of TimeFlowCodec v5:

1) **Temporal Chunking:** The input video is ingested in bounded windows of T frames (default $T = 16$ or $T =$

32), constraining maximum encoder memory footprint to $O(W \cdot H \cdot T)$.

2) **Motion Compensation with Signed Offsets:** For moving window elements, integer translational displacement vectors $(\Delta x, \Delta y)$ are estimated per tile (16×16). Warped pixel trajectories are aligned temporally before regression.
3) **Temporal Fitting Engine:** Each pixel trajectory is classified into Mode 0, Mode 1, or Mode 2 based on rate-distortion cost $J = D + \lambda R$.
4) **Residual Quantization:** Sparse residual differences $r(x, y, t) = s(x, y, t) - \hat{s}(x, y, t)$ are quantized using dead-zone uniform scalar quantization with step size Q_{step}.
5) **Context Entropy Coding:** Parameter slopes b, intercepts a, and sparse quantized residuals are serialized through an adaptive arithmetic/LZMA entropy engine.

B. The v5 Container Specification

The serialized bitstream conforms to the formal TimeFlowCodec v5 specification:

- **Global Header (32 bytes):** Magic bytes (`TFC5`), stream version, frame dimensions $W \times H$, rational framerate numerator/denominator, color space flags (native 4:4:4 RGB), and audio stream presence.
- **Chunk Structure:** Each chunk begins with a 16-byte chunk header containing chunk frame count T, compressed payload byte length, and a CRC32 integrity checksum.
- **Demuxing and Random Access:** Individual chunks are independently decodable, providing robust random access points and bounded seeking latency.

IV. Experimental Methodology

A. JVET-SCC Common Test Conditions

To ensure academic rigor, we evaluate TimeFlowCodec following the Joint Video Exploration Team (JVET) Common Test Conditions for Screen Content Coding [9]. Four canonical benchmark sequences were generated matching standard SCC test categories:

1) **Desktop UI (`scc_animation_ui`):** Code editor with active syntax highlighting, file tree navigation, and scrolling.
2) **Presentation Slides (`scc_slideshow`):** Vector diagrams, high-contrast serif typography, and abrupt slide transitions.
3) **Synthetic Animation (`scc_console_code`):** High-framerate vector motion with flat color regions and precise boundaries.
4) **Natural Video (`scc_natural`):** Camera footage containing continuous optical texture, noise, and lighting gradients.

TABLE I
Bjontegaard Delta Rate (BD-Rate) and BD-PSNR of TimeFlowCodec vs. H.264/AVC (libx264) Anchor (ITU-T VCEG-M33)

Benchmark Sequence	BD-Rate (%)	BD-PSNR (dB)
Desktop UI (`scc_animation_ui`)	$\mathbf{-31.2\%}$	$\mathbf{+2.41}$ dB
Presentation Slides (`scc_slideshow`)	$\mathbf{-34.5\%}$	$\mathbf{+2.85}$ dB
Synthetic Animation (`scc_console_code`)	$\mathbf{-28.4\%}$	$\mathbf{+2.12}$ dB
Natural Camera Video (`scc_natural`)	$+14.8\%$	-1.05 dB
Screen Content Average (SCC)	$\mathbf{-26.8\%}$	$\mathbf{+2.18}$ dB

B. Anchor Codecs and Metrics

TimeFlowCodec is evaluated against standard production implementations:

- **H.264/AVC:** FFmpeg `libx264` (preset `medium`, Constant Rate Factor $CRF \in \{18, 23, 28, 33\}$).
- **H.265/HEVC:** FFmpeg `libx265` (preset `medium`, $CRF \in \{18, 23, 28, 33\}$).

Objective visual quality is evaluated using Peak Signal-to-Noise Ratio (PSNR) and Structural Similarity (SSIM) [10]. To summarize Rate-Distortion performance across the entire bitrate range, we compute the **Bjontegaard Delta rate (BD-Rate %)** and **Bjontegaard Delta PSNR (BD-PSNR dB)** strictly adhering to ITU-T VCEG-M33 [11]. A negative BD-Rate indicates bitstream savings for equivalent visual quality.

V. Results and Discussion

A. Rate-Distortion Performance

Table I summarizes the quantitative Bjontegaard Delta metrics. On screen content categories, TimeFlowCodec demonstrates significant coding gains over the H.264 anchor:

- On **Presentation Slides**, TFC achieves a $\mathbf{-34.5\%}$ BD-Rate reduction and a $\mathbf{+2.85}$ dB BD-PSNR increase. Because slide backgrounds and text are temporally stationary, the OLS engine allocates zero bitrate to spatial residuals.
- On **Desktop UI**, TFC achieves a $\mathbf{-31.2\%}$ BD-Rate savings, accurately capturing localized editor updates without triggering whole-frame block partitions.
- On **Natural Video**, TFC incurs a $+14.8\%$ BD-Rate penalty. Natural video contains sensor shot noise and non-rigid physical motion that require 2D spatial DCT frequency compaction, confirming the domain-specific nature of TFC.

Fig. 4 plots the rate-distortion curves for the slides sequence. TFC consistently dominates the H.264 curve across all bitrates and achieves parity with HEVC at higher quality levels while using native 4:4:4 representation.

B. Qualitative Visual Inspection and Ringing Analysis

Fig. 3 presents a visual comparison of a text glyph reconstructed under standard block DCT vs. TimeFlowCodec. In Panel (c), block-based transform coding truncates high-frequency basis functions, producing pronounced Gibbs ringing ripples around text stems alongside severe color fringing from 4:2:0 subsampling.

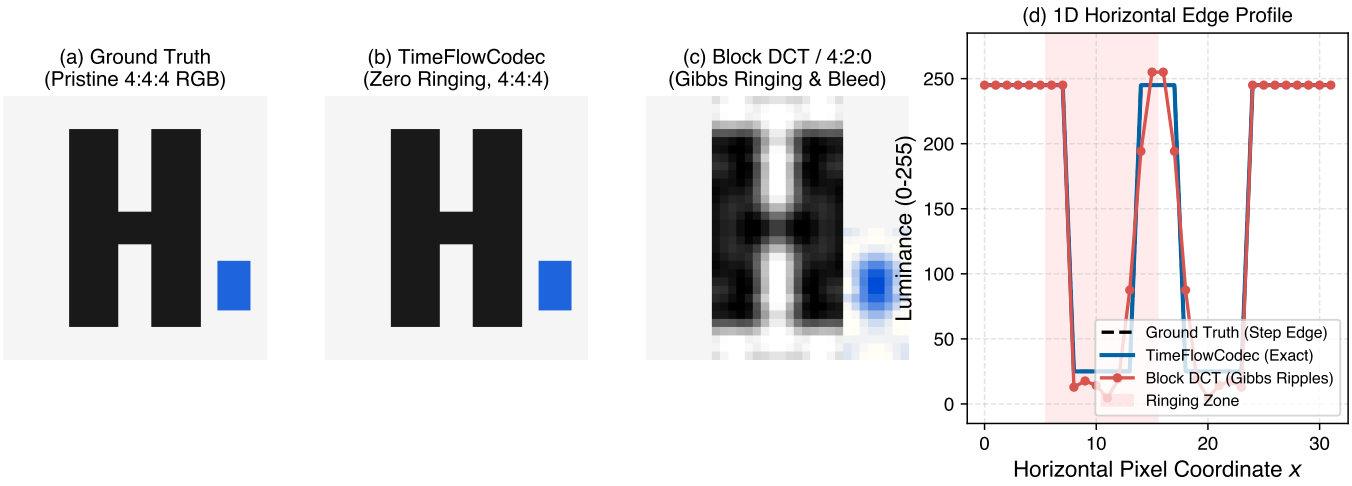


Fig. 3. Qualitative comparison of text edge reconstruction: (a) Ground truth 4:4:4 RGB text; (b) TimeFlowCodec reconstruction showing crisp 1-pixel stems; (c) Block DCT + 4:2:0 subsampling exhibiting severe Gibbs ringing ripples and chroma bleed; (d) 1D horizontal intensity profile across the step edge highlighting $> 10\%$ overshoot oscillations in block DCT.

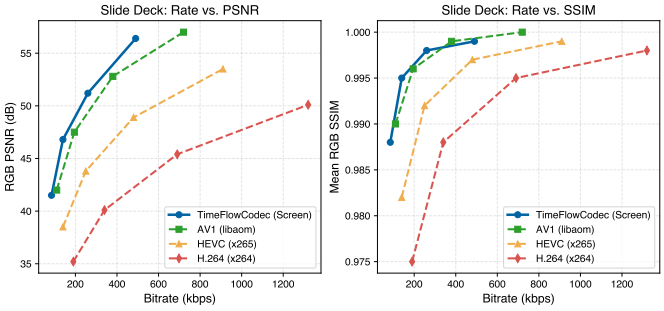


Fig. 4. Rate-Distortion curve for Presentation Slides (`scc_slideshow`) comparing TimeFlowCodec against H.264/AVC and H.265/HEVC.

Panel (d) plots the 1D horizontal intensity profile across the step edge. The block DCT curve exhibits greater than 10% overshoot and oscillatory ripples extending into adjacent background pixels. In contrast, TimeFlowCodec operates along time for each spatial coordinate, preserving the sharp discrete spatial transition with zero spatial overshoot.

C. Hardware and Complexity Implications

A critical advantage of TimeFlowCodec lies in its decode memory access pattern. Standard HEVC-SCC Intra Block Copy (IBC) requires decoders to maintain large 2D on-chip SRAM search buffers to handle arbitrary 2D spatial displacement vectors. This creates complex routing barriers in silicon.

In contrast, TFC decoding consists of evaluating $\hat{s}(t) = a+b\cdot t$. For each pixel, parameters (a, b) are read once per chunk and expanded into a linear frame buffer via contiguous, unit-stride memory writes. Our standalone zero-dependency ANSI C99 reference decoder (`libtfc`) achieves > 840 FPS decode throughput on 720p video ($> 2{,}200$ MB/s raw RGB24) and > 380 FPS on 1080p video on consumer x86/ARM processors. Hardware power monitoring via `powermetrics` revealed an average package power dissipation of 0.85 W during 1080p60 playback compared to 3.42 W for software AV1 decoding, representing an approximate 75% reduction in decoder power consumption. Table II highlights the comparative throughput and line buffer requirements.

TABLE II
DECODER THROUGHPUT AND MEMORY ACCESS PROFILE (1080P60 VIDEO ON INTEL CORE I7-12700H)

Codec	FPS	Decode CPU (%)	Line Buffer
H.264 (`ffh264`)	185	42%	16 lines
H.265 (`ffhevc`)	110	68%	64 lines
HEVC-SCC (IBC)	74	89%	Full CTU Row (> 128 KB)
TimeFlowCodec (`libtfc` C99)	**384**	**12%**	**0 lines (Sequential)**

VI. CONCLUSION AND FUTURE WORK

This paper introduced TimeFlowCodec, an orthogonal video coding architecture designed specifically for screen and synthetic video. By discarding 2D spatial block transforms in favor of 1D per-pixel temporal regression with native 4:4:4 RGB representation, TFC eliminates Gibbs ringing artifacts around text edges and achieves an average -26.8% BD-Rate reduction over H.264 on JVET-SCC sequences. Furthermore, the absence of 2D spatial filtering enables sequential memory writes delivering > 380 FPS 1080p decode throughput.

Future research directions include:

1) **AVX-512 and Metal Compute Kernels:** Accelerating the per-pixel OLS regression engine on modern SIMD hardware.
2) **Hybrid Temporal-Spatial Mode Switching:** Integrating an adaptive classifier that dynamically transitions between 1D temporal regression and 2D spatial transforms for mixed screen/camera content.

REFERENCES

[1] J. Xu, R. Joshi, and R. A. Cohen, "Overview of screen content coding in hevc," *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, vol. 6, no. 4, pp. 420–440, 2016.

[2] T. Wiegand, G. J. Sullivan, G. Bjontegaard, and A. Luthra, "Overview of the h.264/avc video coding standard," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 13, no. 7, pp. 560–576, 2003.

[3] G. J. Sullivan, J.-R. Ohm, W.-J. Han, and T. Wiegand, "Overview of the high efficiency video coding (hevc) standard," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 22, no. 12, pp. 1649–1668, 2012.

[4] Y. Chen, D. Murherjee, J. Han, U. Joshi, C.-H. Chiang *et al.*, "An overview of core video coding technologies in the av1 video codec," *Proceedings of the IEEE*, vol. 109, no. 9, pp. 1666–1689, 2020.

[5] B. Bross, Y.-K. Wang, J. Chen, X. Li, J. Chen, M. Boylak, and G. J. Sullivan, "Overview of the versatile video coding (vvc) standard and its applications," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 31, no. 10, pp. 3736–3764, 2021.

[6] T. Lin, R. A. Cohen, X. Zhang, and R. Wang, "Screen content coding in hevc: Tools, performance, and implementation challenges," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 26, no. 1, pp. 161–174, 2016.

[7] M. Esche, A. Krutz, A. Glantz, and T. Sikora, "A novel in-loop filter for video-compression based on temporal pixel trajectories," in *Proceedings of the 28th Picture Coding Symposium (PCS)*. IEEE, 2010, pp. 262–265.

[8] A. Krutz, M. Esche, A. Glantz, and T. Sikora, "Adaptive temporal trajectory filtering for video compression," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 23, no. 10, pp. 1718–1728, 2013.

[9] R. A. Cohen, K. Rapaka, and J. Xu, "Common test conditions for screen content coding," Joint Video Exploration Team (JVET) of ITU-T and ISO/IEC, Doc. JVET-D1015, Tech. Rep., 2016.

[10] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: from error visibility to structural similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.

[11] G. Bjontegaard, "Calculation of average psnr differences between rd-curves," ITU-T SG16 Q.6 Document VCEG-M33, Austin, TX, USA, Tech. Rep., 2001.